

Java Constructor Chaining — Complete Beginner to Advanced Guide

You already know what a constructor is.

Now we'll learn one of the most important Java concepts:

Constructor Chaining

This is heavily used in:

- Selenium frameworks
- Playwright frameworks
- Appium frameworks
- Spring Boot
- Dependency Injection
- Page Object Model (POM)

1. What is Constructor Chaining?

Constructor chaining means:

One constructor calling another constructor.

Java allows constructors to call:

- another constructor of SAME class → using `this()`
- constructor of PARENT class → using `super()`

This creates a **chain of constructor execution**.

Simple Example

```
class Student {  
  
    Student() {  
        this("Iqbal");  
        System.out.println("Default Constructor");  
    }  
  
    Student(String name) {  
        System.out.println("Name Constructor: " + name);  
    }  
  
    public static void main(String[] args) {  
        new Student();  
    }  
}
```

Output

Name Constructor: Iqbal
Default Constructor

Flow

```
Student()  
|  
---> this("Iqbal")  
    |  
    ---> Student(String name)
```

This is constructor chaining.

2. Types of Constructor Chaining

There are mainly 2 types:

Type	Keyword
Same class constructor chaining	this()
Parent class constructor chaining	super()

E. Complex Confidence Building Example

Realistic Banking Example

```
class BankAccount {  
  
    String name;  
    int balance;  
    String branch;  
  
    BankAccount() {  
        this("Unknown");  
        System.out.println("Default Constructor");  
    }  
  
    BankAccount(String name) {  
        this(name, 0);  
        System.out.println("Single Argument Constructor");  
    }  
  
    BankAccount(String name, int balance) {  
        this(name, balance, "Bangalore");  
        System.out.println("Two Argument Constructor");  
    }  
}
```

```
BankAccount(String name, int balance, String branch) {  
  
    this.name = name;  
    this.balance = balance;  
    this.branch = branch;  
  
    System.out.println("Final Constructor");  
}  
  
void display() {  
    System.out.println(name);  
    System.out.println(balance);  
    System.out.println(branch);  
}  
  
public static void main(String[] args) {  
  
    BankAccount b = new BankAccount();  
  
    b.display();  
}  
}
```

Output Flow

Final Constructor

Two Argument Constructor

Single Argument Constructor

Default Constructor

Unknown

0

Bangalore

4. Why Chaining and use of this() and super()?

Without chaining:

```
Employee() {  
    name = "Iqbal";  
    age = 25;  
}
```

```
Employee(String name) {  
    this.name = name;  
    age = 25;  
}
```

Repeated code.

With chaining

```
Employee() {  
    this("Iqbal");  
}
```

```
Employee(String name) {  
    this(name, 25);  
}
```

```
Employee(String name, int age) {  
    this.name = name;  
    this.age = age;  
}
```

Benefits:

- less duplicate code
- centralized initialization
- clean framework design
- maintainability

5. Difference Between this Keyword and this()

Feature	this	this()
Refers to current object	YES	NO
Calls constructor	NO	YES
Used inside methods	YES	NO
Used inside constructors	YES	YES

Example of this

```
class Student {  
  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    void show() {  
        System.out.println(this.name);  
    }  
  
    public static void main(String[] args) {  
  
        Student s = new Student("Iqbal");  
  
        s.show();  
    }  
}
```

```
}  
}
```

Example of this()

```
class Student {  
  
    Student() {  
        this("Iqbal");  
    }  
  
    Student(String name) {  
        System.out.println(name);  
    }  
  
    public static void main(String[] args) {  
        new Student();  
    }  
}
```

6. Real-Time Use in Automation Framework

Very important.

Example — Selenium Framework

```
class BasePage {  
  
    WebDriver driver;
```

```
BasePage(WebDriver driver) {  
    this.driver = driver;  
}  
}
```

Login Page

```
class LoginPage extends BasePage {  
  
    LoginPage(WebDriver driver) {  
  
        super(driver);  
  
        System.out.println("Login Page Created");  
    }  
}
```

Why super(driver)?

Because:

- driver belongs to parent class
- child page reuses same driver
- avoids duplicate code

Used in:

- Selenium
- Playwright
- Appium
- POM
- API frameworks

7. Common Mistakes Java Programmers Do

Mistake 1 — Infinite Constructor Recursion

```
class Test {  
  
    Test() {  
        this();  
    }  
}
```

✗ StackOverflowError

Mistake 2 — this() not on first line

```
Test() {  
  
    System.out.println("Hello");  
  
    this(10);  
}
```

✗ Compile error

Mistake 3 — using both super() and this()

```
Test() {  
  
    this(10);  
  
    super();  
}
```

✗ Not allowed

Mistake 4 — Duplicate Initialization

Bad framework design.

How to avoid

- Keep one final constructor
- Use chaining smartly
- Avoid circular constructor calls
- Use inheritance carefully

8. What if we use this() on second line?

```
class Test {  
  
    Test() {  
  
        System.out.println("Hello");  
    }  
}
```

```
        this(10);  
    }  
  
    Test(int a) {  
  
    }  
}
```

✗ Compile Error

Constructor call must be first statement

Why?

Java needs constructor chain resolved FIRST before object initialization.

9. Where can we use this()?

ONLY inside constructors.

Can we call this() inside method?

✗ NO

```
void show() {  
    this();  
}
```

Compile error.

Why?

Because constructor invocation is allowed only during object creation.

10. What if we use multiple statements in constructor?

Perfectly fine.

```
class Test {  
  
    Test() {  
  
        int a = 10;  
  
        System.out.println(a);  
  
        System.out.println("Constructor");  
    }  
  
    public static void main(String[] args) {  
  
        new Test();  
    }  
}
```

But this() or super() MUST remain first

Correct:

```
Test() {
```

```
this(10);

System.out.println("Hello");
}
```

11. What if we try recursion on constructor invocation?

Example

```
class Test {

    Test() {
        this(10);
    }

    Test(int a) {
        this();
    }
}
```

Flow

```
Test()
-> Test(int)
   -> Test()
      -> Test(int)
```

Infinite loop.

Result:

StackOverflowError

12. Why Constructor Chaining with Different Class?

Because inheritance needs parent initialization first.

Example

```
class Animal {  
  
    Animal() {  
        System.out.println("Animal Constructor");  
    }  
}  
  
class Dog extends Animal {  
  
    Dog() {  
        System.out.println("Dog Constructor");  
    }  
  
    public static void main(String[] args) {  
        new Dog();  
    }  
}
```

Output

```
Animal Constructor  
Dog Constructor
```

Why?

Parent should initialize before child.

13. super() Statement

super() calls parent constructor.

Example

```
class Parent {  
  
    Parent() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
  
    Child() {  
  
        super();  
  
        System.out.println("Child");  
    }  
  
    public static void main(String[] args) {  
  
        new Child();  
    }  
}
```

14. What if super() is second line?

```
class Child extends Parent {  
  
    Child() {  
  
        System.out.println("Hello");  
  
        super();  
    }  
}
```

✗ Compile error

Constructor call must be first statement

15. What if we have multiple super()?

```
class Child extends Parent {  
  
    Child() {  
  
        super();  
  
        super();  
    }  
}
```

✗ Compile error

Why?

Only ONE parent constructor can be directly called.

16. Default Constructor Extends Object

Every Java class internally extends:

$A \rightarrow Object$

Equivalent:

```
class Test extends Object {  
  
}
```

Why?

Because Object class gives:

- toString()
- equals()
- hashCode()
- wait()
- notify()

All Java classes inherit these.

17. Drama Between super() and this()

Both want FIRST line.

But Java allows only ONE first statement.

So this is illegal:

```
Test() {  
  
    this(10);  
  
    super();  
}
```

Important Rule

If you use:

```
this()
```

Eventually some constructor in chain MUST call:

```
super()
```

directly or indirectly.

18. Explain class A extends Object

```
class A {  
  
}
```

Compiler converts internally:

```
class A extends Object {  
  
}
```

Why Object is Necessary?

Because Java wants:

- common root
- polymorphism
- memory management
- universal methods

19. Real-Time Examples for Automation Engineer

Selenium Page Object Model

```
class BasePage {  
  
    WebDriver driver;  
  
    BasePage(WebDriver driver) {  
        this.driver = driver;  
    }  
}
```

Child Page

```
class DashboardPage extends BasePage {  
  
    DashboardPage(WebDriver driver) {  
  
        super(driver);  
    }  
}
```

Why this design?

Benefits:

- single driver object
- reusable methods
- inheritance
- clean architecture

Appium Framework Example

```
class BaseTest {  
  
    AndroidDriver driver;  
  
    BaseTest(AndroidDriver driver) {  
        this.driver = driver;  
    }  
}
```

API Framework Example

```
class BaseAPI {  
  
    RequestSpecification request;  
  
    BaseAPI(RequestSpecification request) {  
        this.request = request;  
    }  
}
```

20. Very Important Interview Questions

Q1

Can constructor be final?

 NO

Q2

Can constructor be static?

 NO

Q3

Can constructor be private?

 YES

Singleton pattern.

Q4

Can constructor return value?

 NO

Q5

Can we inherit constructor?

✗ NO directly

But child can call parent constructor using `super()`.

Most Important Rules to Remember

Rule	Valid
<code>this()</code> must be first line	✓
<code>super()</code> must be first line	✓
both together	✗
<code>this()</code> inside method	✗
recursive constructor call	✗
multiple <code>super()</code>	✗

Final Understanding

Constructor chaining is mainly for:

- object initialization order
- avoiding duplicate code
- inheritance initialization
- clean framework architecture

In automation frameworks:

- `super(driver)`
- dependency injection
- page initialization

- base class reuse

are everywhere.

Once you master constructor chaining:

- inheritance becomes easy
- framework design improves
- Spring/Selenium/Appium architecture becomes understandable